

HelixTranslate

HelixTranslate

Проверенная модель перевода книг — честная по замыслу, без скрытых откатов.

Краткое описание

HelixTranslate — высокопроизводительная платформа для перевода электронных книг на базе Go, работающая с более чем 100 языками. Использует только проверенных поставщиков LLM, обеспечивает мониторинг в реальном времени (WebSocket) и строгую политику отказа без скрытых резервных вариантов: система либо завершает работу с явной ошибкой, либо не деградирует незаметно.

Краткая характеристика

Универсальный инструмент для перевода электронных книг на базе Go. Поддерживает форматы FB2, EPUB, TXT, HTML, PDF и DOCX, переводит на более чем 100 языков с использованием лучших проверенных моделей LLM (через мост LLMsVerifier). Оснащён API REST/HTTP-3 и gRPC, распределённой обработкой данных и панелью мониторинга WebSocket в реальном времени.

Подробное описание

HelixTranslate — это система корпоративного уровня на базе Go, предназначенная для перевода книг целиком между языками с использованием поставщиков LLM. Речь идёт не о переводе отдельных абзацев или фрагментов, а о полноценных книжных произведениях от начала до конца. Система обрабатывает и генерирует несколько форматов электронных книг (FB2, EPUB, TXT, HTML, PDF, DOCX), поддерживает более 100 языков с автоматическим определением исходного языка, а также предлагает как инструменты командной строки (CLI),

так и серверные решения (API) — REST поверх HTTP/3, gRPC и поток событий WebSocket. Это позволяет интегрировать платформу как в рабочий процесс через терминал, так и в сервисную инфраструктуру.

Ключевая особенность системы — *принцип выбора модели*: вместо жёсткого закрепления за одним поставщиком в надежде, что он останется работоспособным, HelixTranslate делегирует все полномочия по выбору модели мосту LLMsVerifier (pkg/bridge). Он отбирает наиболее надёжную *проверенную* модель API и формирует детерминированную цепочку откатов, упорядоченную по рейтингу. Пригодность модели определяется взвешенной оценкой по таким критериям, как скорость отклика, качество кода, функциональное богатство и надёжность. Таким образом, модель, выполняющая ваш перевод, занимает своё место не потому, что её прописали в конфигурационном файле, а потому, что она доказала свою эффективность на практике.

Крайне важно, что система жёстко реализует принцип «никаких скрытых откатов» прямо на уровне кода: если ключ поставщика API отсутствует или оператор явно запрашивает недоступного поставщика, конвейер возвращает явную критическую ошибку, а не переключается на другого поставщика или не переходит на локальный движок, создавая иллюзию благополучия. Это правило закреплено специальным предсборочным гейтом и парным мутационным тестом. Локальные движки (Ollama, llama.cpp) намеренно исключены из стандартного пути выполнения, чтобы более слабый механизм никогда не подменял собой проверенную модель.

Вокруг ядра перевода функционирует подсистема мониторинга WebSocket в реальном времени: инструмент перевода (CLI) генерирует типизированные события, которые поступают на сервер мониторинга и отображаются на живой веб-панели. Удалённые рабочие процессы SSH распределяют нагрузку для параллельного перевода. Дополнительно реализованы многоэтапная полировка для обеспечения согласованности, предварительный анализ качества на этапе подготовки, кэширование переводов для снижения затрат при работе с длинными текстами, а также визуальный контроль качества. Вся платформа подчиняется принципам антиблефовой инженерной культуры: тесты должны подтверждать реальные, видимые пользователю результаты, подкреплённые обязательным мутационным тестированием, а не просто «зелёными галочками», которые ничего не доказывают.

Почему мы это создали

Переводить объёмные книги надёжно и *честно* — никогда не поставляя «ухудшенный, но хоть какой-то» перевод. Основной принцип: отсутствие или невозможность верификации перевода должно быть явной, жёсткой ошибкой, а выбор модели — всегда приводить к действительно проверенному поставщику, а не к жёстко заданному предположению или тихому локальному откату.

Почему это меняет правила игры

Большинство конвейеров перевода LLM молча проваливаются — они незаметно переключаются на более слабую модель, сбрасываются на локальный рантайм или выдают частичный результат, в то время как тесты гордо светятся зелёным, а никто даже не замечает обвала качества. HelixTranslate делает такой сценарий структурно невозможным: выбор модели контролируется верификацией, цепочка откатов детерминирована и полностью прозрачна, а «нет ключа / нет проверенной модели» приводит к честной критической ошибке, а не к молчаливому пожатию плечами. Это единственное проектное решение превращает вопрос «А действительно ли этот перевод выполнен на проверенной, работоспособной модели?» из надежды, которую не проверить, в гарантию, которую система обеспечивает за вас.

Что здесь нового

- **Маршрутизация моделей с контролем верификации** через мост LLMsVerifier — автоматически выбирается самая сильная *проверенная* модель, так что операторы задают намерение, а не имена вендоров, и никогда не выбирают вручную поставщика, который может оказаться недоступен.
- **Гарантия отсутствия тихих откатов, закреплённая в коде** — четыре явных ветви маршрутизации (мок / явный верификатор / явный поставщик / мост по умолчанию), каждая из которых жёстко завершается ошибкой, а не молча переключается, плюс намеренное удаление локальных рантаймов из пути по умолчанию, чтобы не на что было откатываться.
- **Механическое обеспечение** — предсборочный гейт CM-NO-LOCAL-RUNTIME и парный мутационный тест проверяют на этапе сборки, что клиент локального рантайма никогда не создаётся на пути по умолчанию: гарантия не может устареть, потому что сборка падает, если это происходит.

- **Детерминированная цепочка откатов с упорядочиванием по качеству** — разрешены и полностью прозрачны переключения между *проверенными* моделями, что принципиально отличается от запрещённых тихих откатов: вы всегда знаете, какая работоспособная модель взяла на себя задачу.
- **Мониторинг WebSocket в реальном времени** — типизированные события перевода транслируются в прямом эфире на дашборд, с распределёнными воркерами SSH, так что работа над книгой видна и параллельна, а не остаётся чёрным ящиком.
- **Режим антиблефа в тестировании** — мутационное тестирование, негативные утверждения, запуски на реальных системах и QA с визуальной проверкой вместе гарантируют, что «тесты прошли» никогда не маскирует «функция на самом деле не работает».

Главные технические вызовы и как мы их преодолели

- **Гарантия честного конвейера перевода (без молчаливой деградации).** Решено путём централизации всех полномочий по выбору модели в мосте LLMsVerifier, чтобы был единственный контролируемый узел принятия решений, кодирования четырёх явных ветвей маршрутизации, каждая из которых завершается ошибкой вместо предположений, полного удаления локальных откатов из пути по умолчанию и закрепления правила с помощью предсборочного гейта и мутационного теста, который падает, если гарантия когда-либо будет нарушена.
- **«Зелёные тесты, сломанные функции».** В конституции этот сценарий провала назван напрямую и преодолен с помощью режима антиблефа в тестировании: конкретные, видимые пользователю утверждения вместо технических деталей реализации, реальные системы в цикле (моки только в юнит-тестах), обязательное мутационное тестирование (намеренное нарушение функции должно приводить к падению теста) и визуально проверяемый QA, который действительно смотрит на результат.
- **Качество при работе с длинными текстами и несколькими форматами.** Книги большого объёма создают нагрузку как на согласованность, так и на бюджет; решено с помощью многоэтапной полировки, которая возвращается к тексту, предварительного анализа, оценивающего объём работы, и кэширования переводов, чтобы не платить дважды за один и тот же фрагмент.

Технологический стек

- **Go** — выбран за примитивы конкурентности, которые естественным образом ложатся на задачи параллельного парсинга, перевода и потоковой передачи множества глав; высококонкурентный бэкенд, модуль `digital.vasic.translator`.
- **Gin** — выбран как быстрый и минималистичный HTTP-маршрутизатор для обслуживания поверхности REST API.
- **QUIC / HTTP/3 (quic-go)** — выбраны для обеспечения низкой задержки и современной транспортной инфраструктуры REST API, устойчивой к нестабильным сетям.
- **gRPC + Protocol Buffers** — выбраны для создания строго типизированного высокопроизводительного сервисного интерфейса, работающего параллельно с REST для программных клиентов.
- **Gorilla WebSocket** — выбран для передачи потока событий перевода в реальном времени с типизацией, который питает живую панель мониторинга.
- **PostgreSQL, SQLite, Redis** — осознанное трёхступенчатое разделение: PostgreSQL для долговременных реляционных данных, SQLite для локального/встроенного состояния (также используется как хранилище проверенных моделей моста, `data/verified_models.db`), а Redis — как горячий кэш.
- **unidoc/unioffice + unipdf** — выбраны для работы со сложными форматами: парсинг и регенерация DOCX и PDF, чтобы мультимедийные электронные книги корректно проходили полный цикл обработки.
- **Cobra** — выбран как фреймворк CLI, обеспечивающий работу `unified-translator` и сопутствующих инструментов.
- **golang-jwt (JWT HS256)** — выбран для бесстатусной аутентификации API в сочетании с порейтинговым ограничением токенов на основе IP и транспортной безопасностью TLS/QUIC для усиления защиты поверхности.
- **Мост LLMsVerifier (pkg/bridge)** — ключевой элемент: предоставляет наиболее надёжную проверенную модель и её детерминированную цепочку откатов, а также служит единственной точкой контроля для гарантии отсутствия скрытых откатов.
- **Testify** — выбран для тестового набора Go, включая специализированный `provider_routing_test.go` и шлюзы мутаций, которые обеспечивают соблюдение правил честности.
- **Docker / Podman (rootless) + Compose** — выбраны для контейнеризованного распределённого развёртывания (`docker-compose.distributed.yml`), с rootless-режимом Podman для повышения уровня безопасности.

Статус и замечания о честности

- **Статус: бета-версия.** Платформа функциональна; версия указана непоследовательно в файлах VERSION/Makefile/AGENTS.md, поэтому считается неустоявшейся.
- **Лицензия: не определена.** В README заявлена MIT, но это не подтверждено наличием файла LICENSE — требуется проверка перед утверждением.
- Конечные точки панели мониторинга доступны только на localhost и не публичны. Показатели производительности WebSocket в документации приведены как целевые, а не подтверждённые. В ARCHITECTURE.md всё ещё упоминаются удалённые локальные движки Ollama (устаревшие данные).

Приоритетный уровень: Helix-primary (инфраструктурный кластер LLM). Занимает место в семействе платформ Helix после HelixTrack.